📘 Modules in Python

Importing Modules | 'import' vs 'from' statement | math Module | random Module | statistics Module | All Built-in Functions

Introduction to Python Modules

Definition: A module is a file containing Python code (functions, classes, variables) that can be imported and used in other Python programs. Modules help organize code into reusable components.

```
# A module is simply a .py file
# Example: mymodule.py contains functions we can reuse
# Python has many built-in modules like math, random, statistics
```

Important Points:

- Modules promote code reusability and organization
- Python has a rich standard library of built-in modules
- Modules must be imported before they can be used
- Importing a module gives access to all its functions and variables
- Common built-in modules: `math`, `random`, `statistics`, `os`, `sys`, `datetime`

Importing Modules

1. Using import <module>

Purpose: Imports an entire module, making all its functions accessible

Syntax: `import module_name`

Parameters: module_name (compulsory) — name of the module to import

Return: None (makes module available in the program)

- Access functions using `module_name.function_name()`
- Imports the entire module
- Keeps the namespace clean (functions are prefixed with module name)

```
import math

print(math.pi)           # 3.141592653589793
print(math.sqrt(16))     # 4.0
print(math.ceil(4.2))    # 5
```

2. Using from <module> import <function>

Purpose: Imports specific functions from a module directly

Syntax: `from module_name import function_name`

Parameters: module_name (compulsory), function_name (compulsory or *)

Return: None (makes specified functions available directly)

- Access functions directly without module prefix
- Can import multiple functions: `from math import pi, sqrt`
- Can import all functions using `*` : `from math import *`
- May cause naming conflicts if function names overlap

```
from math import pi, sqrt, ceil

print(pi)           # 3.141592653589793
print(sqrt(25))     # 5.0
print(ceil(4.1))    # 5

# Import all functions
from math import *
print(floor(4.9))   # 4
```

3. Using import <module> as <alias>

Purpose: Imports a module with a shorter alias name

Syntax: `import module_name as alias`

Parameters: module_name (compulsory), alias (compulsory)

Return: None (makes module available with alias)

- Useful for modules with long names
- Access functions using `alias.function_name()`
- Common aliases: `import numpy as np` , `import pandas as pd`

```
import math as m

print(m.pi)         # 3.141592653589793
print(m.sqrt(36))   # 6.0
```

Math Module

Import: `import math`

1. math.pi

Purpose: Mathematical constant pi (π)

Syntax: `math.pi`

Parameters: None (constant)

Return: Float (3.141592653589793)

- Ratio of circumference to diameter of a circle

```
import math
print(math.pi)  # 3.141592653589793
```

2. math.e

Purpose: Mathematical constant e (Euler's number)

Syntax: `math.e`

Parameters: None (constant)

Return: Float (2.718281828459045)

- Base of natural logarithms

```
import math
print(math.e)  # 2.718281828459045
```

3. math.sqrt()

Purpose: Returns the square root of a number

Syntax: `math.sqrt(x)`

Parameters: x (compulsory) — non-negative number

Return: Float (square root of x)

- Raises ValueError if x is negative
- x can be integer or float

```
import math
print(math.sqrt(16))    # 4.0
print(math.sqrt(2))     # 1.4142135623730951
```

4. math.ceil()

Purpose: Rounds a number up to the nearest integer

Syntax: `math.ceil(x)`

Parameters: x (compulsory) — number to round up

Return: Integer (smallest integer greater than or equal to x)

- Always rounds up (towards positive infinity)
- ceil(4.1) = 5, ceil(-4.1) = -4

```
import math
print(math.ceil(4.1))    # 5
print(math.ceil(4.9))    # 5
print(math.ceil(-4.1))   # -4
```

5. math.floor()

Purpose: Rounds a number down to the nearest integer

Syntax: `math.floor(x)`

Parameters: x (compulsory) — number to round down

Return: Integer (largest integer less than or equal to x)

- Always rounds down (towards negative infinity)
- floor(4.9) = 4, floor(-4.1) = -5

```
import math
print(math.floor(4.1))    # 4
```

```
print(math.floor(4.9))    # 4
print(math.floor(-4.1))   # -5
```

6. math.pow()

Purpose: Returns x raised to the power y

Syntax: `math.pow(x, y)`

Parameters: x (compulsory) — base; y (compulsory) — exponent

Return: Float (x raised to the power y)

- Always returns a float
- Equivalent to x ** y but returns float

```
import math
print(math.pow(2, 3))    # 8.0
print(math.pow(5, 2))    # 25.0
```

7. math.fabs()

Purpose: Returns the absolute value of a number

Syntax: `math.fabs(x)`

Parameters: x (compulsory) — number

Return: Float (absolute value of x)

- Always returns a float
- Similar to built-in abs() but returns float

```
import math
print(math.fabs(-5))     # 5.0
print(math.fabs(5))      # 5.0
print(math.fabs(-3.14))  # 3.14
```

8. math.sin()

Purpose: Returns the sine of an angle (in radians)

Syntax: `math.sin(x)`

Parameters: x (compulsory) — angle in radians

Return: Float (sine of x, between -1 and 1)

- Angle must be in radians, not degrees
- Use `math.radians()` to convert degrees to radians

```
import math
print(math.sin(0))                # 0.0
print(math.sin(math.pi/2))        # 1.0
print(math.sin(math.radians(30))) # 0.49999999999999994
```

9. math.cos()

Purpose: Returns the cosine of an angle (in radians)

Syntax: `math.cos(x)`

Parameters: x (compulsory) — angle in radians

Return: Float (cosine of x, between -1 and 1)

- Angle must be in radians, not degrees
- Use `math.radians()` to convert degrees to radians

```
import math
print(math.cos(0))                # 1.0
print(math.cos(math.pi))          # -1.0
print(math.cos(math.radians(60))) # 0.5000000000000001
```

10. math.tan()

Purpose: Returns the tangent of an angle (in radians)

Syntax: `math.tan(x)`

Parameters: x (compulsory) — angle in radians

Return: Float (tangent of x)

- Angle must be in radians, not degrees
- Use `math.radians()` to convert degrees to radians

```
import math
print(math.tan(0))                # 0.0
```

```
print(math.tan(math.pi/4))      # 0.9999999999999999
print(math.tan(math.radians(45))) # 0.9999999999999999
```

Random Module

Import: `import random`

1. random.random()

Purpose: Returns a random floating-point number between 0.0 and 1.0

Syntax: `random.random()`

Parameters: No parameters

Return: Float (0.0 <= x < 1.0)

- 0.0 is included, 1.0 is excluded
- Useful for generating random probabilities

```
import random
print(random.random())  # 0.5488135039273248 (varies)
print(random.random())  # 0.7151893663724195 (varies)
```

2. random.randint()

Purpose: Returns a random integer between two specified values (inclusive)

Syntax: `random.randint(a, b)`

Parameters: a (compulsory) — lower bound; b (compulsory) — upper bound

Return: Integer (a <= x <= b)

- Both endpoints are included
- a must be less than or equal to b

```
import random
print(random.randint(1, 10))   # 7 (varies, between 1 and 10)
print(random.randint(1, 6))    # 3 (varies, like a dice roll)
```

3. random.randrange()

Purpose: Returns a random integer from a range (start, stop, step)

Syntax: `random.randrange(start, stop, step)`

Parameters:

- start (optional) — lower bound (default: 0)
- stop (compulsory) — upper bound (excluded)
- step (optional) — step value (default: 1)

Return: Integer (start <= x < stop, with step)

- Similar to `range()` but returns a random number
- stop is excluded

```
import random
print(random.randrange(10))       # 0-9 (varies)
print(random.randrange(1, 10))    # 1-9 (varies)
print(random.randrange(0, 10, 2)) # 0, 2, 4, 6, 8 (varies)
```

Statistics Module

Import: `import statistics`

1. statistics.mean()

Purpose: Returns the arithmetic mean (average) of numeric data

Syntax: `statistics.mean(data)`

Parameters: data (compulsory) — list, tuple, or iterable of numbers

Return: Float (arithmetic mean)

- Sum of all values divided by the number of values
- Raises StatisticsError if data is empty

```
import statistics
data = [10, 20, 30, 40, 50]
print(statistics.mean(data))  # 30.0
```

2. statistics.median()

Purpose: Returns the median (middle value) of numeric data

Syntax: `statistics.median(data)`

Parameters: data (compulsory) — list, tuple, or iterable of numbers

Return: Float or Integer (middle value)

- If odd number of values: middle value
- If even number of values: average of two middle values
- Raises StatisticsError if data is empty

```
import statistics
data1 = [10, 20, 30, 40, 50]
print(statistics.median(data1))  # 30

data2 = [10, 20, 30, 40]
print(statistics.median(data2))  # 25.0
```

3. statistics.mode()

Purpose: Returns the mode (most frequently occurring value) of numeric data

Syntax: `statistics.mode(data)`

Parameters: data (compulsory) — list, tuple, or iterable of numbers

Return: The most frequent value

- Returns the first mode if multiple modes exist
- Raises StatisticsError if data is empty

```
import statistics
data = [10, 20, 20, 30, 20, 40]
print(statistics.mode(data))  # 20

data2 = [1, 2, 2, 3, 3, 4]
print(statistics.mode(data2))  # 2 (first mode)
```

Quick Reference Table

Module	Function / Constant	Purpose	Returns
`math`	`pi`	Mathematical constant π	Float
	`e`	Euler's number	Float
	`sqrt(x)`	Square root	Float
	`ceil(x)`	Round up	Integer
	`floor(x)`	Round down	Integer
	`pow(x, y)`	x raised to y	Float
	`fabs(x)`	Absolute value	Float
	`sin(x)`	Sine (radians)	Float
	`cos(x)`	Cosine (radians)	Float
	`tan(x)`	Tangent (radians)	Float
`random`	`random()`	Random float [0.0, 1.0)	Float
	`randint(a, b)`	Random integer [a, b]	Integer
	`randrange(s, e, st)`	Random integer from range	Integer
`statistics`	`mean(data)`	Arithmetic average	Float
	`median(data)`	Middle value	Float / Integer
	`mode(data)`	Most frequent value	Element

Important Board Exam Tips Must Know

1. **Importing Modules:**
   - `import math` — access with `math.sqrt()`
   - `from math import sqrt` — access directly with `sqrt()`
   - `import math as m` — access with `m.sqrt()`
2. **math Module Key Points:**
   - `pi` and `e` are constants (no parentheses)

- `ceil()` rounds up, `floor()` rounds down
- `sqrt()` raises ValueError for negative numbers
- `pow()` and `fabs()` always return floats
- Trigonometric functions take radians, not degrees
- Use `math.radians(degrees)` to convert

3. **random Module Key Points:**
   - `random()` returns float between 0.0 and 1.0 (1.0 excluded)
   - `randint(a, b)` includes both endpoints
   - `randrange(start, stop, step)` excludes stop
   - `randint(1, 6)` simulates a dice roll
4. **statistics Module Key Points:**
   - `mean()` = sum / count
   - `median()` = middle value (average of two middle values if even count)
   - `mode()` = most frequently occurring value
   - All three raise StatisticsError on empty data
5. **Common Errors to Avoid:**
   - Forgetting to import the module before using its functions
   - Using `math.sqrt(-4)` — raises ValueError
   - Passing degrees to `math.sin()` instead of radians
   - Using `random.randint(10, 1)` — a must be <= b
   - Confusing `random()` with `randint()`
   - Forgetting that `randrange()` excludes the stop value
6. **Board Exam Favourites:**
   - Write a program to find the area of a circle using `math.pi`
   - Generate a random number between 1 and 100 using `random.randint()`
   - Calculate mean, median, and mode of a list of numbers
   - Differentiate between `import math` and `from math import *`
